

《OpenCV + Python: 图像与视频处理基础》

授课教师：王洪松

邮箱：hongsongsui@gmail.com

目标跟踪

目标跟踪：Meanshift

<https://hongsong-wang.github.io/>

```
import numpy as np
import cv2

cap = cv2.VideoCapture('slow.flv')
# take first frame of the video
ret,frame = cap.read()
# setup initial location of window
r,h,c,w = 250,90,400,125 # simply hardcoded the values
track_window = (c,r,w,h)
# set up the ROI for tracking
roi = frame[r:r+h, c:c+w]
hsv_roi = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv_roi, np.array((0., 60.,32.)), np.array((180.,255.,255.)))
roi_hist = cv2.calcHist([hsv_roi],[0],mask,[180],[0,180])
cv2.normalize(roi_hist,roi_hist,0,255,cv2.NORM_MINMAX)
# Setup the termination criteria, either 10 iteration or move by atleast 1 pt
term_crit = ( cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 1 )
```

目标跟踪: Meanshift

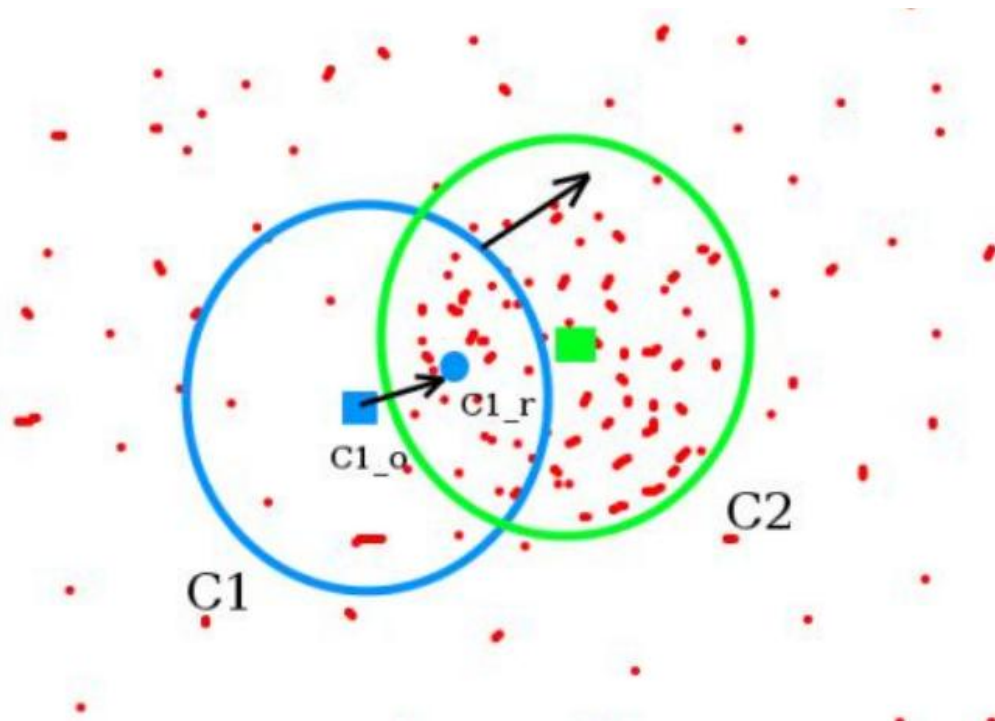
<https://hongsong-wang.github.io/>

```
while(1):
    ret, frame = cap.read()
    if ret == True:
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        dst = cv2.calcBackProject([hsv],[0],roi_hist,[0,180],1)
        # apply meanshift to get the new location
        ret, track_window = cv2.meanShift(dst, track_window, term_crit)
        x,y,w,h = track_window # Draw it on image
        img2 = cv2.rectangle(frame, (x,y), (x+w,y+h), 255,2)
        cv2.imshow('img2',img2)
        k = cv2.waitKey(60) & 0xff
        if k == 27:
            break
        else:
            cv2.imwrite(chr(k)+".jpg",img2)
    else:
        break
cv2.destroyAllWindows()
cap.release()
```



- Meanshift 算法的原理

- 假设有一堆点（比如直方图反向投影得到的点）， 和一个小的圆形窗口。
- 将这个窗口移动到最大灰度密度处（或者是点最多的地方）。



初始窗口是蓝色的“C1”，它的圆心为蓝色方框“C1_o”，而窗口中所有点质心却是C1_r”，很明显圆心和点的质心没有重合。所以移动圆心 C1_o 到质心 C1_r，这样我们就得到了一个新的窗口。这时又可以找到新窗口内所有点的质心，大多数情况下还是不重合的。所以重复上面的操作：将新窗口的中心移动到新的质心。

- Meanshift 目标跟踪的窗口的大小是固定的，对于运动目标，固定的窗口是不合适的。
- Camshift根据目标的大小和角度来对窗口的大小和角度进行修订：
 - 首先要使用 meanshift，找到目标之后，再调整窗口的大小。
 - 计算目标对象的最佳外接椭圆的角度，并以此调节窗口角度。
 - 重复以上步骤直到达到需要的精度。

```
# apply meanshift to get the new location
ret, track_window = cv2.CamShift(dst, track_window, term_crit)
# Draw it on image
pts = cv2.boxPoints(ret)
pts = np.int0(pts)
img2 = cv2.polylines(frame,[pts],True, 255,2)
```

- 相关滤波（Correlation Filtering）是利用滤波器模板记录并更新目标信息，并通过计算模板与每帧图像各位置的相似度来输出响应图，取响应最高的位置即最相似的位置作为目标位置。
- MOSSE (Minimum Output Sum of Squared Error filter)

希望有一个滤波器 h ，使得 $g = f * h$

根据傅里叶变换，空域卷积等于频域乘积 $G = F \odot H^*$

G 和 F 分别表示 g 和 f 的傅里叶变换， H^* 表示 H 的复共轭。注意因为相关和卷积在某些地方可能当成相同的运算，实际上两者是有差别的， f 与 h 做相关实际上应该是 f 与旋转了180度的 h 做卷积，这也是为什么是 H^* 而不是 H 的原因。

G 和 F 都已知，则可以得到： $H^* = \frac{G}{F}$

- MOSSE

要适用于不同的f，则H*应该满足： $\min_{H^*} \sum_i |F_i \odot H^* - G_i|^2$

对H*求导，令其为0，可得到闭式解： $H^* = \frac{\sum_i G_i \odot F_i^*}{\sum_i F_i \odot F_i^*}$

将第一帧图像的目标进行多次的变换，得到多个f，计算初始化的H*

为了能够利用后续视频帧对H*进行更新，MOSSE给出如下方法：

$$H_i^* = \frac{A_i}{B_i}$$

$$A_i = \eta G_i \odot F_i^* + (1 - \eta) A_{i-1}$$

$$B_i = \eta F_i \odot F_i^* + (1 - \eta) B_{i-1}$$

η 其中表示学习率， η 越大，则对当前帧保留的信息越多，对历史信息保留的越， η 最佳值为0.125。

- Meanshift、MOSSE等目标跟踪方法需要在起始帧选择一个窗口，然后在随后的帧自动跟踪该目标。这类方法叫做单目标跟踪：
 - 算法一次只跟踪一个目标
 - 第一帧需要选择目标框，其它帧算法自动生成预测框
- 多目标跟踪：跟踪属于一个或多个类别的多个目标（如行人、车辆、动物等），同时保持每个目标的身份一致性。

SORT

<https://github.com/abewley/sort>

Deep SORT

https://github.com/nwojke/deep_sort/tree/master

- 数字跟踪
 - 跟踪视频中的所有数字，用矩形框标出目标的位置，同一个目标用同一个数字ID表示，ID起始于0。
 - 相同ID对应的矩形框用相同颜色，不同ID用不同颜色，矩形框的附近注明ID的编号。
- 行人跟踪
 - 第一帧用鼠标选择多个行人目标框。
 - 采用MOSSE或其它算法同时对多个目标框进行自动跟踪，要求修改mosse代码或其它算法的代码，以支持多个目标同时跟踪，并尽量减少重复计算，保证代码的运行效率（假如跟踪 n 个目标， $n=1,2,\dots$ ，计算跟踪所需的时间，画出该时间和 n 的曲线）。

- 快速标注视频车辆
 - 给定一个车辆视频，如何用矩形框快速且准确标记出视频中的所有车辆。
 - 不同帧，相同的车辆用同一个ID，不同车辆ID不同。
 - 人工每帧标记需要大量时间，用算法自动检测并跟踪车辆，存在检测框不准、漏检或误检、跟踪错误等问题。